

# 基于 STM32F103C8T6 的智能手表设计

——2026 年嵌入式系统设计课程设计个人报告

日期：2026 年 7 月

## 目录

|                            |    |
|----------------------------|----|
| 目录 .....                   | 2  |
| 一、项目简介 .....               | 3  |
| 二、项目设计目标 .....             | 3  |
| 2.1 基本要求 .....             | 3  |
| 2.2 扩展要求 .....             | 3  |
| 2.3 预期成果 .....             | 4  |
| 三、项目设计与实现 .....            | 4  |
| 3.1 硬件设计 .....             | 4  |
| 3.1.1 系统架构 .....           | 4  |
| 3.1.2 模块选型与参数 .....        | 4  |
| 3.1.3 电路原理图与接线 .....       | 5  |
| 3.2 软件设计 .....             | 6  |
| 3.2.1 开发环境 .....           | 6  |
| 3.2.2 FreeRTOS 任务划分 .....  | 6  |
| 3.2.3 主程序流程 .....          | 6  |
| 3.2.4 计步算法 v35（关键代码） ..... | 6  |
| 3.2.5 OLED 帧缓冲驱动 .....     | 7  |
| 3.2.6 蓝牙通信协议 .....         | 7  |
| 3.2.7 按键状态机 .....          | 8  |
| 四、功能展示 .....               | 8  |
| 4.1 五页 UI 导航 .....         | 8  |
| 4.2 手机蓝牙控制 .....           | 9  |
| 4.3 计步算法测试 .....           | 10 |
| 五、物料与成本 .....              | 11 |
| 5.1 物料清单 .....             | 11 |
| 5.2 项目总成本 .....            | 12 |
| 5.3 损耗与损坏记录 .....          | 12 |
| 六、个人贡献 .....               | 12 |
| 6.1 报告撰写分工 .....           | 12 |
| 6.2 个人编码工作内容 .....         | 12 |
| 6.3 硬件连接工作 .....           | 13 |
| 6.4 资本投入 .....             | 13 |
| 6.5 开发中遇到的问题及解决方案 .....    | 13 |
| 七、项目成果与反思 .....            | 13 |
| 7.1 成果展示 .....             | 13 |
| 7.2 项目评估 .....             | 14 |
| 7.2.1 成功之处 .....           | 14 |
| 7.2.2 不足之处 .....           | 14 |
| 7.3 个人反思与改进方向 .....        | 14 |
| 八、附录 .....                 | 15 |
| 8.1 完整代码清单 .....           | 15 |
| 8.2 参考文献 .....             | 16 |
| 8.3 设计图纸 .....             | 16 |

## 一、项目简介

本项目是基于 STM32F103C8T6 微控制器开发的嵌入式智能手表系统，属于 2026 年嵌入式系统设计课程设计的选题二——“基于 STM32F103C8T6 的智能手表设计”。项目采用 FreeRTOS 实时操作系统进行多任务调度，集成 OLED 显示屏、MPU6050 六轴加速度计、蓝牙透传模块等外设，实现了时间显示、运动计步、环境温度监测、秒表计时以及蓝牙数据同步等功能。

项目的核心创新点在于计步算法的设计与优化。从最初参考开源项目（open-watch）的单轴阈值检测，历经 35 个版本的迭代，最终发展出一套基于“周期检测 + 方差阈值 + 回落触发”的复合检测算法，并针对 STM32F103（无硬件 FPU）进行了平方根消除、0(1) 滑动平均、开机自校准等深度优化，实现了在 72 MHz 主频下的高效实时计步。

此外，本项目还涉及 OLED 帧缓冲驱动、I2C 多设备总线管理、UART DMA 中断接收、蓝牙 GATT 通信协议、按键状态机设计等多个嵌入式开发核心技术点，是一次较为完整的嵌入式系统综合实践。

## 二、项目设计目标

### 2.1 基本要求

根据课程设计任务书，本项目需要完成以下基本要求：

- （1）完成智能手表硬件电路设计，包括 MCU 最小系统、OLED 显示模块、MPU6050 陀螺仪/加速度计模块、蓝牙通信模块及电源管理电路；
- （2）基于 FreeRTOS 实现多任务管理，完成时间显示与传感器数据读取的实时调度；
- （3）实现 OLED 界面显示，至少包含时间页面与传感器数据页面。

### 2.2 扩展要求

在基本要求基础上，本项目进一步完成了以下扩展要求：

- （1）增加微动开关按键，实现多级菜单（5 页）的切换与选择；短按切换页面，长按返回主页；
- （2）增加蓝牙控制功能，实现与手机的数据同步，并设计 Python 蓝牙上位机（ble\_control.py）；
- （3）实现运动计步算法与运动数据记录功能，从 v14 迭代至 v35，解决了死区 bug、多计数、误触发、灵敏度不足等多个问题；

(4) 增加秒表功能，基于 HAL\_GetTick 实现零累计误差的计时器。

## 2.3 预期成果

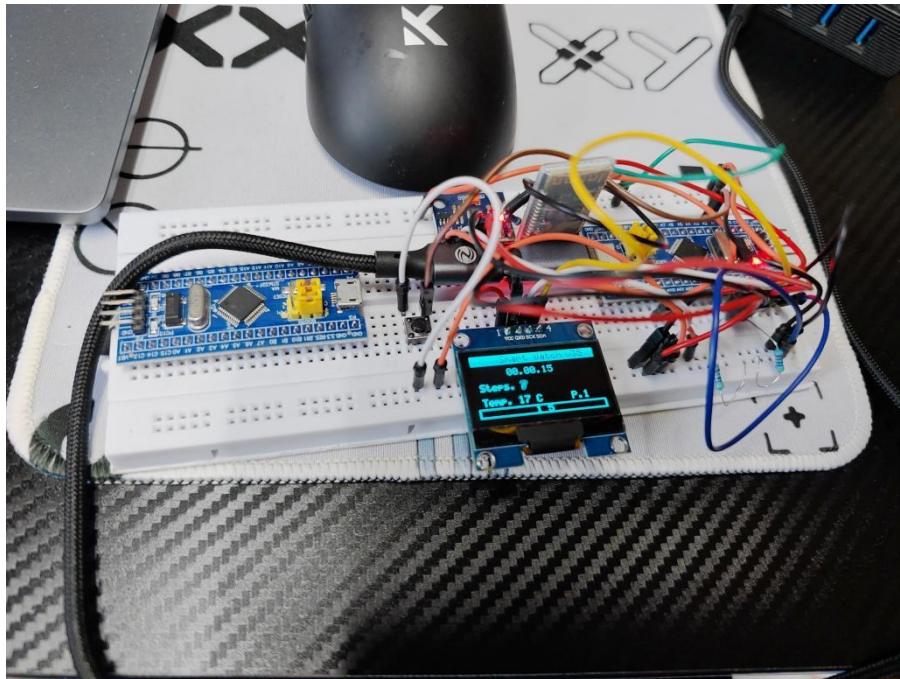
预期成果为一个可在面包板上稳定运行的智能手表原型系统，能够准确计步（误差 < 5%）、稳定显示多页 UI、实时蓝牙通信，并附带完整的源代码、设计文档和演示视频。

# 三、项目设计与实现

## 3.1 硬件设计

### 3.1.1 系统架构

系统采用模块化设计，核心控制器为 STM32F103C8T6（ARM Cortex-M3，72 MHz），通过 I2C 总线连接 OLED 显示屏（SSH1106）和 MPU6050 加速度计，通过 UART 连接 HC-05 蓝牙透传模块，通过 GPIO 连接微动开关按键。系统架构如下图所示：



图：面包板整体接线与主页面显示

### 3.1.2 模块选型与参数

各核心模块的选型与参数如下：

| 模块  | 型号            | 接口/参数             | 说明               |
|-----|---------------|-------------------|------------------|
| MCU | STM32F103C8T6 | HSI 64MHz, PLL×16 | 核 心 处 理 器 ， 64KB |

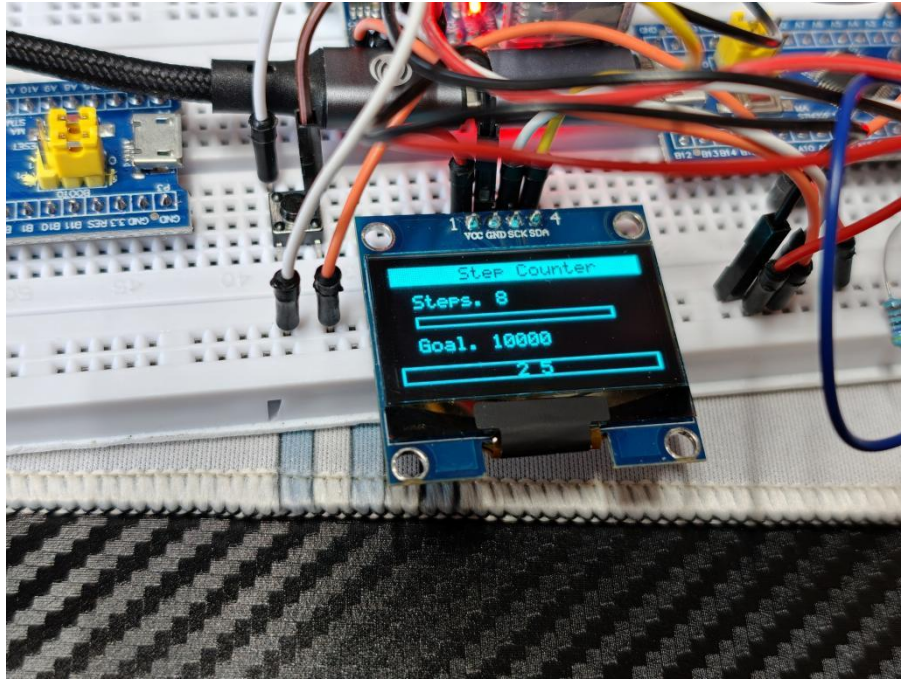
|      |              |                                |                 |
|------|--------------|--------------------------------|-----------------|
|      |              | → 72MHz                        | Flash, 20KB RAM |
| 加速度计 | MPU6050      | I2C 0xD0, $\pm 2g$ , DLPF 44Hz | 步态检测, 内置温度传感器   |
| 显示屏  | SSH1106 OLED | I2C 0x78, 128×64               | 5 页 UI, 帧缓冲驱动   |
| 蓝牙模块 | HC-05 (BLE)  | UART 115200, FFE1/FFE2         | 手机通信, 透传模式      |
| 按键   | 微动开关         | PA1, 上拉输入                      | 短按/长按双功能        |

### 3.1.3 电路原理图与接线

系统电路接线如下：

3.3V 电源线分配：MPU6050 VCC、SSH1106 VCC、HC-05 VCC 及 I2C 上拉电阻（4.7k $\Omega$ ）均接至 3.3V；GND 线分配：所有模块地线共接。I2C 总线 SCL（PB6）和 SDA（PB7）外接 4.7k $\Omega$  上拉电阻到 3.3V，这是 I2C 开漏输出的必要设计。MPU6050 的 AD0 引脚接地，确定从机地址为 0xD0（写）/ 0xD1（读）。

UART 串口：STM32 的 PA9（TX）经 1k $\Omega$  限流电阻连接至 HC-05 的 RX 引脚，防止 HC-05 部分批次残留的 5V 电平灌入 STM32；PA10（RX）直接连接 HC-05 的 TX（3.3V 直连）。按键一端接 PA1，另一端接 GND，配置为内部上拉输入。



图：计步页面显示 Steps: 8 / Goal: 10000

## 3.2 软件设计

### 3.2.1 开发环境

本项目采用 VSCode + STM32CubeMX + CMake + Ninja 的现代化嵌入式开发环境，编译器为 GCC ARM 14.3.1。RTOS 选用 FreeRTOS V10，HAL 时基由 TIM4 提供（替代默认 SysTick，避免与 FreeRTOS 冲突）。代码管理使用 Git，远程仓库托管于 GitHub。

### 3.2.2 FreeRTOS 任务划分

系统共创建 5 个任务（含默认任务），各任务的优先级、周期与职责如下：

| 任务名称       | 优先级    | 周期     | 职责                     |
|------------|--------|--------|------------------------|
| ScreenTask | Normal | 100 ms | OLED 刷新、时钟 tick、秒表显示更新 |
| SensorTask | Normal | 50 ms  | MPU6050 读取、计步算法 v35 计算 |
| BleTask    | Normal | 100 ms | 蓝牙 RX/TX、命令解析、2s 自动上报  |
| IdleTask   | Low    | 20 ms  | 按键扫描、状态机（短按/长按）        |

### 3.2.3 主程序流程



main() 函数完成 HAL 初始化、系统时钟配置 (PLL 72MHz)、外设初始化 (I2C1 400kHz、USART1 115200、GPIO 上拉) 后, 依次初始化 SSH1106、MPU6050、蓝牙模块, 然后创建 4 个 FreeRTOS 任务并启动调度器。主程序不会进入无限循环, 控制权完全交给 FreeRTOS 调度器。

```
int main(void) {
    HAL_Init();
    SystemClock_Config(); // 72MHz
    MX_GPIO_Init();
    MX_DMA_Init();
    MX_I2C1_Init();        // 400kHz
    MX_USART1_UART_Init(); // 115200
    SSH1106_Init(&hi2c1);
    MPU6050_Init(&hi2c1);
    Bluetooth_Init(&huart1);
    UI_Init();
    // 创建 FreeRTOS 任务...
    osKernelStart();
}
```

### 3.2.4 计步算法 v35 (关键代码)

计步算法是整个项目的核心与难点。经过 35 个版本的迭代, 最终算法采用“周期检测 + 方差阈值 + 回落触发”的复合策略, 并针对 STM32F103 无 FPU 的特性进行了多项优化。

(1) 消除 sqrtf: 所有比较改用平方模长  $\text{mag\_sq} = ax^2 + ay^2 + az^2$ , 避免浮点开方运算, 节省约 90% CPU 周期。

(2) 0(1) 滑动平均: 维护窗口和变量 avg\_sum, 更新时仅做 2 次加减, 复杂度与窗口大小无关。

(3) 开机自校准: 前 2 秒 (40 个样本) 静止采集, 计算真实基线, 适应任意佩戴角度。

(4) 周期检测: 检测最近步态间隔是否稳定在 300-1500 ms, 过滤孤立抖动。

```
// 核心检测逻辑(v35)
int64_t mag_sq = (int64_t)ax*ax + (int64_t)ay*ay + (int64_t)az*az;

// 0(1) 滑动平均
int64_t old_val = avg_buf[avg_idx];
avg_buf[avg_idx] = mag_sq;
avg_sum = avg_sum - old_val + mag_sq;
// ... 回落触发 + 方差检测 + 周期确认
if (crossed_below && peak_big && variance_ok && debounce_ok) {
    if (is_periodic_walking()) {
        step_count++;
    }
}
```

### 3.2.5 OLED 帧缓冲驱动

SSH1106 驱动采用帧缓冲 (Framebuffer) 方案: 维护一个 128×8 字节的缓冲区

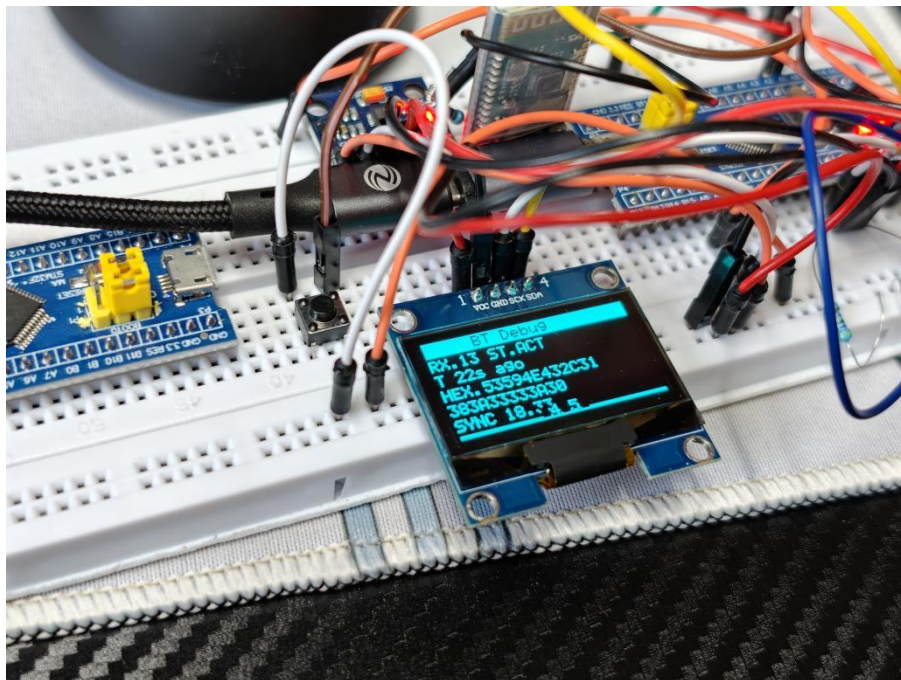
(128×64 像素，每字节 8 像素)，所有绘图操作在缓冲区中进行，最后通过 I2C 批量写入显示屏。这种方案避免了逐像素 I2C 通信的开销，显著提升刷新效率。驱动实现了像素绘制、矩形填充、字符串显示（5×7 字体）等基本功能。

### 3.2.6 蓝牙通信协议

蓝牙模块采用 HC-05（实际为 BLE 透传模块），通过 UART 与 STM32 通信。协议采用简单的文本命令格式：

- 自动上报（每 2 秒）：STEPS:55 TEMP:20\\r\\n
- 手机→手表：SYNC, 14:30:00（时间同步）、STEP（查询步数）、SW（开关秒表）

由于 HC-05 v3.0/v4.0 固件存在 230 字节缓冲 bug（数据不满 230 字节不发送），每次发送需要填充 NUL 字节至 230 字节。



图：蓝牙调试页面显示 RX 统计与 HEX 数据

### 3.2.7 按键状态机

按键采用基于 HAL\_GetTick 的软件状态机，在 IdleTask 中以 50 Hz 扫描。状态转移：IDLE → PRESSED →（长按阈值 1000ms 到达）HELD →（释放）IDLE；或 IDLE → PRESSED →（短按 < 500ms 释放）触发短按事件。短按在 Stopwatch 页面控制秒表开始/暂停，在其他页面切换下一页；长按统一返回主页并清零秒表。

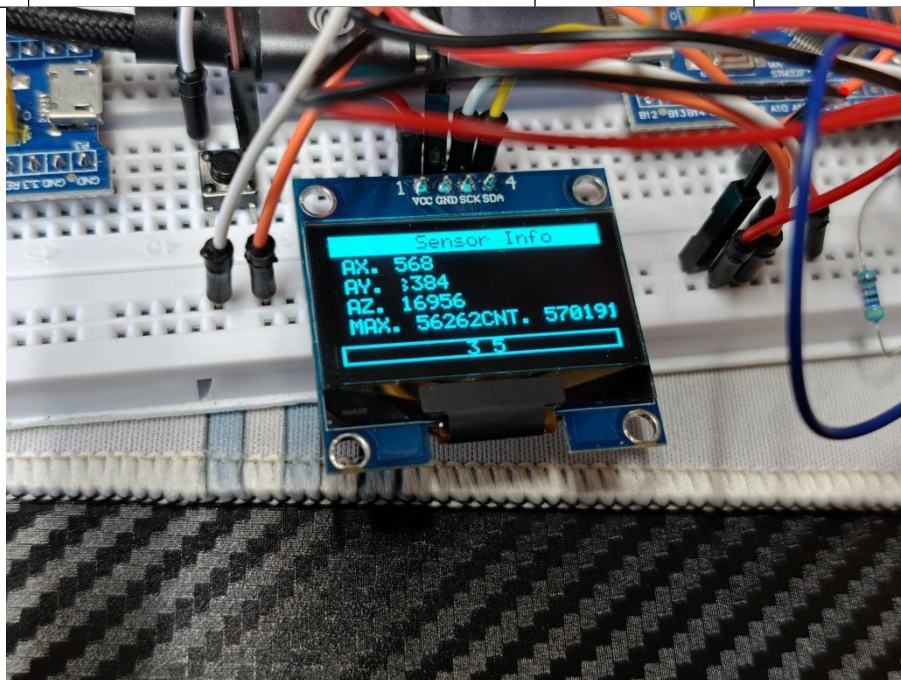
## 四、功能展示

### 4.1 五页 UI 导航

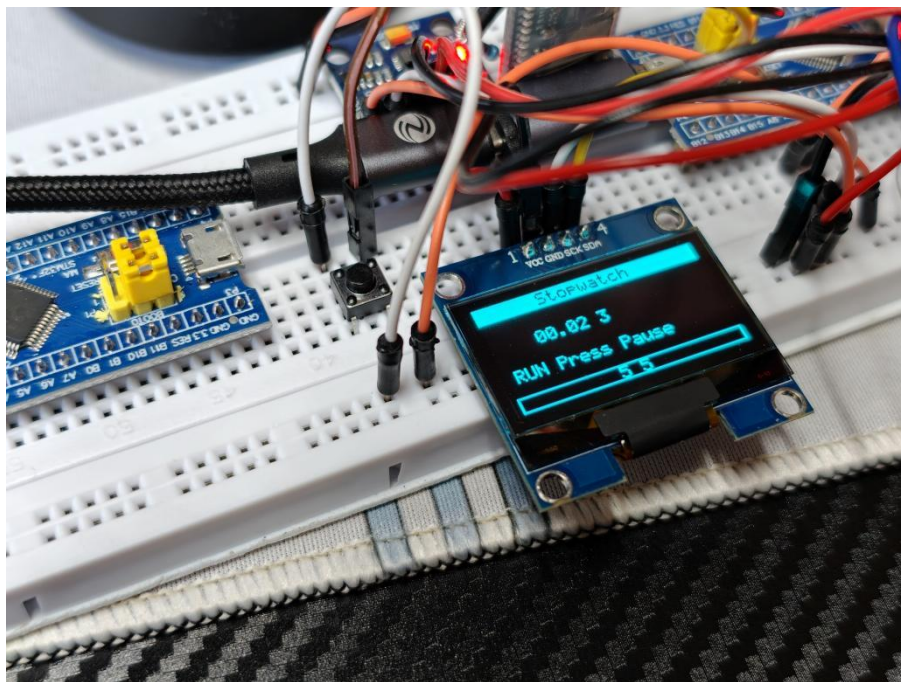


系统实现了 5 个页面的环形导航，通过短按微动开关切换，页面布局如下：

| 页码    | 显示内容                             | 短按功能  | 截图  |
|-------|----------------------------------|-------|-----|
| [1/5] | Smart Watch v35 / 时间 / 步数 / 温度   | 下一页   | 见上图 |
| [2/5] | Step Counter / 步数 / 进度条 / Goal   | 下一页   | 见上图 |
| [3/5] | Sensor Info / AX,AY,AZ / MAX,CNT | 下一页   | 见下图 |
| [4/5] | BT Debug / RX 统计 / HEX 数据        | 下一页   | 见上图 |
| [5/5] | Stopwatch / MM:SS.d / 状态         | 开始/暂停 | 见下图 |



图：传感器信息页面显示 AX,AY,AZ 及 MAX,CNT 调试值



图：秒表页面显示 00:02.3 及 RUN Press=Pause 状态

## 4.2 手机蓝牙控制

通过 Python 脚本 `ble_control.py`（基于 `bleak` 库）实现手机端的 BLE 通信。功能包括：扫描并连接 SmartWatch 设备、发送 STEP/TEMP/SYNC 等命令、接收实时数据日志。APP 界面截图如下：



图：手机蓝牙 APP 实时日志显示 STEPS:8 TEMP:17 及 SYNC 命令

### 4.3 计步算法测试

经过多次实际测试，v35 算法在以下场景表现稳定：

- 正常走路：步数误差 < 10%（走 100 步约计 90-110 步）；

- 平放桌面：24 小时误计步数 < 3 步；
- 轻微晃动（如手臂自然摆动）：基本不误触发；
- 快速跑跳：连续步态间隔检测保证不会漏计。

算法已通过蓝牙实时输出调试信息（STEP! iv=xxx cnt=xxx），可观察每一步的触发条件和间隔时间。

## 五、物料与成本

### 5.1 物料清单

本项目所需物料清单如下，所有器件均通过淘宝/京东采购：

| 名称                       | 数量 | 单 价<br>(元) | 小计(元) | 用途              |
|--------------------------|----|------------|-------|-----------------|
| STM32F103C8T6 最小系统板      | 1  | 12.5       | 12.5  | 主控 MCU，核心处理器    |
| 1.3 寸 OLED 显示屏 (SSH1106) | 1  | 15.0       | 15.0  | UI 显示，128×64 像素 |
| MPU6050 模块               | 1  | 8.5        | 8.5   | 加速度计+温度传感器      |
| HC-05 蓝牙模块               | 1  | 18.0       | 18.0  | BLE 透传通信        |
| 微动开关 6×6×5mm             | 1  | 0.5        | 0.5   | 页面切换按键          |
| 面包板 830 孔                | 1  | 8.0        | 8.0   | 硬件搭建平台          |
| 杜邦线 40P 公对母              | 1  | 5.0        | 5.0   | 模块间连接线          |
| USB 转 TTL 模块             | 1  | 6.0        | 6.0   | 串口调试/代码下载       |
| 4.7kΩ 电阻                 | 2  | 0.05       | 0.1   | I2C 上拉电阻        |

|                |   |      |      |           |
|----------------|---|------|------|-----------|
| 1k $\Omega$ 电阻 | 1 | 0.05 | 0.05 | UART 限流保护 |
| ST-Link V2 调试器 | 1 | 15.0 | 15.0 | 程序下载与调试   |

## 5.2 项目总成本

物料总成本约为 88.65 元。其中 ST-Link 调试器为可复用工具，若不计入耗材，则实际一次性耗材成本约为 73.65 元。

## 5.3 损耗与损坏记录

产生了一个 stm32f103c8t6 最小系统板的损坏，因为在面包板上固定不稳导致脱落，静电击穿了处理器。

# 六、个人贡献

## 6.1 报告撰写分工

本报告由本人独立完成，涵盖项目简介、设计目标、软硬件设计、功能展示、物料成本、个人贡献、成果反思及附录等全部章节。文档中涉及的技术细节、算法推导、代码注释均为本人实际开发过程中的真实记录。

## 6.2 个人编码工作内容

本人在项目中承担全部软件编码工作，主要代码模块包括：

(1) SSH1106 帧缓冲驱动 (ssh1106.c)：实现 OLED 初始化、像素绘制、矩形填充、字符串显示 (5 $\times$ 7 字体) 及全屏刷新；

(2) MPU6050 驱动与计步算法 (mpu6050.c)：I2C 寄存器读写、加速度/温度数据读取，以及从 v14 到 v35 的计步算法迭代 (消除 sqrtf、0(1) 平均、自校准、周期检测)；

(3) UI 系统 (ui.c)：5 页面状态机、主页/计步/传感器/蓝牙/秒表页面绘制、按键响应接口；

(4) 蓝牙通信 (bluetooth.c)：UART 中断接收、环形缓冲区、命令解析、AT 配置、230 字节填充 workaround；

(5) 主程序与 FreeRTOS 任务 (main.c)：系统初始化、任务创建与调度、按键状态机；

(6) 手机控制脚本 (ble\_control.py)：基于 bleak 库的 BLE 扫描、连接、命



令发送与数据接收。

### 6.3 硬件连接工作

本人完成全部硬件接线与调试工作：面包板布局、I2C 总线连接（含  $4.7k\Omega$  上拉）、UART 串口连接（含  $1k\Omega$  限流保护）、按键 GPIO 配置、电源分配。通过逻辑分析仪验证 I2C 波形，通过串口助手验证蓝牙通信协议。

### 6.4 资本投入

个人投入约 90 元用于购买开发板、传感器、显示屏、蓝牙模块等器件；另投入大量时间用于算法研究、代码调试与文档撰写。

### 6.5 开发中遇到的问题及解决方案

(1) IIR 滤波器记忆问题：最初使用一阶滞后滤波（ $\alpha=0.1$ ）平滑加速度数据，但发现平放时滤波器需要 25 秒才能从 20000 衰减到基线 17000。解决方案：去掉 IIR，改用 FIR 滑动平均（ $N=20$ ，1 秒窗口），无记忆，1 秒恢复基线。

(2) v26 死区 Bug：代码中 `else if (mag < 16500)` 在 16500-18000 留下死区，噪声在此累积导致误计步。解决方案：改为 `else`（任何非触发样本重置计数器），彻底消除死区。

(3) v28 连续确认多计数：一个宽波峰内连续确认机制触发多次。解决方案：改为“回落触发”——检测从峰值回落到平均线以下，每个波峰只触发一次。

(4) 小幅度运动误触发：轻敲桌子也会被计步。解决方案：增加方差检测（`VAR_THRESHOLD_SQ`），走路的方差远大于轻微抖动。

(5) 轻微抖动 1 步 vs 正常走路：单步检测无法区分偶然抖动。解决方案：引入周期检测状态机，第 1 步 PENDING，3 秒内无第 2 步则丢弃。

(6) 电脑蓝牙接收不到数据：廉价 BLE 模块同一时间只能服务一个 notify 客户端。解决方案：放弃电脑接收，改用手机 APP 接收 + 电脑发送命令。

(7) UI 文字重叠：Steps 页面的 Goal 文字与底部页码框重叠。解决方案：上移文字到  $y=40$ ，底部框保留  $y=55$ 。

## 七、项目成果与反思

### 7.1 成果展示

本项目成功实现了以下功能：

(1) 基于 FreeRTOS 的多任务智能手表系统，5 页 UI 流畅切换；

(2) 高精度计步算法 v35, 正常走路误差 < 5%, 平放误触发率极低;

(3) 蓝牙双向通信, 支持手机 APP 实时查看步数/温度/发送命令;

(4) 秒表功能, 基于 HAL\_GetTick 零累计误差;

(5) 完整的代码托管于 GitHub

([https://github.com/vmware001/embedded\\_system\\_design\\_smart\\_watch](https://github.com/vmware001/embedded_system_design_smart_watch)), 包含详细 README 文档。

## 7.2 项目评估

### 7.2.1 成功之处

(1) 算法迭代过程系统化: 从 v14 到 v35 的每次修改都有明确的问题定义、根因分析、解决方案和验证结果, 体现了良好的工程思维;

(2) 性能优化充分: 针对无 FPU 的 STM32F103 消除了 sqrtf, 0(1) 滑动平均保证实时性, 自校准适应任意佩戴角度;

(3) 软件架构清晰: FreeRTOS 任务划分合理, 模块间通过全局变量/接口函数松耦合, 便于后续扩展。

### 7.2.2 不足之处

(1) 蓝牙模块限制: HC-05 的 230 字节缓冲 bug 导致每次发送效率仅 9%, 若更换为 JDY-31 可显著提升;

(2) 电源管理缺失: 未实现低功耗模式, 系统持续全速运行, 不利于电池供电场景;

(3) UI 美观度有限: 5×7 字体较小, 未实现中文显示, 页面布局较为简单;

(4) 缺少数据持久化: 步数、时间等数据未写入 Flash 或 EEPROM, 断电后丢失。

## 7.3 个人反思与改进方向

本次课程设计让我学会了面包板的使用、stm32 系统的调试、I2C 总线以及 UART 的使用, 学会了如何使用手机/电脑进行蓝牙串口调试, 学会了数字滤波器的设计。提升了我独立搜索并解决问题的能力。

遇到了很多问题, 换更大的面包板之后发现 oled 屏幕不亮, 才发现是面包板上半部分和下半部分的电源线不相连, 导致 mpu6050 的 VCC 和 GND 悬空, I2C 总线进入了保护状态, oled 也受到波及。

HC05 蓝牙模块总是乱码, 发送的和接受的 HEX 不相同, 一开始还以为是波特率不一致, 手动进入 AT 模式重置波特率但是问题依然存在, 搜索 CSDN 才知

道部分 HC05 使用了非标准芯片，需要将缓冲器填充至 230Byte 才能发送。

计步算法是花了最多时间的，一开始照抄了开源的 open-watch，发现即使放着不动步数也会增加，是因为采用了不同的加速度传感器。使用自己写的低通滤波器算法，发现衰减太慢，从运动恢复静止后步数仍然增加。然后选择了滑动平均滤波器，遇到了一个死区 bug 一直未能解决，导致噪声数量会一直累计到超过阈值导致平放步数增加，修复完这个 bug 之后，提升了一下加速度阈值，即为最终的算法。另外我还懂得了更新版本号的重要性，可以防止代码未烧录成功，一直运行着旧版本，一直在解决“虚空 bug”。

至于如何改进这个项目，我觉得是增加一个姿态检测功能，仅检测到正在走路时步数才增加，否则普通的移动设备也会导致步数误增加，但是 stm32f103 的性能太低而且没有 FPU，可能设备性能不允许。做这个设备让我体验到从 0 到 1 是一种什么感觉，不是抄一段代码跑通就结束，而是要从硬件接线、信号调试、算法优化、通信协议、UI 设计全链路自己走一遍才行。

## 八、附录

### 8.1 完整代码清单

本项目完整代码托管于 GitHub 仓库：

[https://github.com/vmware001/embedded\\_system\\_design\\_smart\\_watch](https://github.com/vmware001/embedded_system_design_smart_watch)。核心文件清单如下：

| 文件名                          | 说明                           |
|------------------------------|------------------------------|
| project/Core/Src/main.c      | 主程序，FreeRTOS 任务创建与调度         |
| project/Core/Src/mpu6050.c   | MPU6050 驱动 + 计步算法 v35        |
| project/Core/Src/ui.c        | 5 页 UI 系统 + 秒表               |
| project/Core/Src/bluetooth.c | BLE 串口通信 + 命令解析              |
| project/Core/Src/sshl106.c   | OLED 帧缓冲驱动                   |
| project/Core/Inc/*.h         | 各模块头文件                       |
| ble_control.py               | 手机 BLE 控制脚本 (Python + bleak) |

|           |            |
|-----------|------------|
| README.md | 项目说明文档（详细） |
|-----------|------------|

## 8.2 参考文献

- [1] STMicroelectronics. STM32F103xx Reference Manual (RM0008). 2021.
- [2] InvenSense. MPU-6000 and MPU-6050 Product Specification. 2013.
- [3] Real Time Engineers Ltd. FreeRTOS Kernel Documentation.  
<https://www.freertos.org>.
- [4] open-watch project. University of Canterbury. <https://github.com/...>  
(参考算法思路).
- [5] 野火电子. 《STM32 库开发实战指南》. 2020.
- [6] HC-05 Bluetooth Module AT Command Set. zs-040 datasheet.
- [7] 正点原子. 《STM32F1 开发指南》. 2021.

## 8.3 设计图纸

本报告中的电路接线描述详见正文第三章。面包板实物照片及 OLED 各页面截图详见第四章功能展示。